

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Hemanth Kumar R
24011101035
B.Tech AI&DS 'A'

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

The preprocessing pipeline produced the following input tensor shapes:

Input tensor shape:

Training : (6054, 128, 9)

Validation : (1298, 128, 9)

Testing : (2947, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

The training set contains 6054 sequences, the validation set contains 1298 sequences, and the independent test set contains 2947 sequences. Each sequence consists of 128 time steps and 9 sensor channels.

6. Temporal Data Visualization

Representative sequences were selected from three different activity classes: **WALKING**, **WALKING UP-STAIRS**, and **SITTING**. For each activity, the Body Acceleration X, Y and Z signals were plotted over the 128 time steps.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Channels: Body Acceleration X, Y and Z
- Activities: WALKING, WALKING UPSTAIRS and SITTING

Plot 1: Temporal Sensor Signals Across Activities



Inference:

1. **Temporal pattern:** WALKING and WALKING UPSTAIRS exhibit periodic variations in the acceleration signals owing to the continuous body movement whereas SITTING exhibits less variations compared to the former.

2. **Similar activities:** WALKING and WALKING UPSTAIRS exhibit similarity since both of them are activities of locomotion whereas SITTING exhibits a different pattern since there is no continuous locomotion activity.
3. **Ordering:** Ordering of 128 samples is important since it indicates how sensor values vary with time.

7. Vanilla RNN Architecture

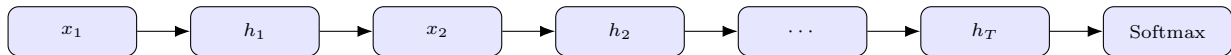
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

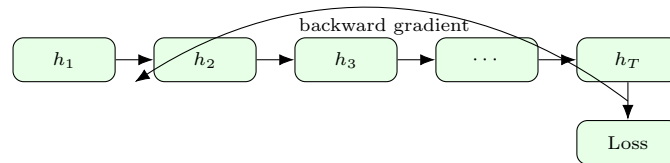
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

the hidden states are calculated as follows:

$$h_1 = \tanh(0.5(0.5) + 0.8(0) + 0.1) = \tanh(0.35) \approx 0.3364$$

$$h_2 = \tanh(0.5(0.7) + 0.8(0.3364) + 0.1) \approx 0.6164$$

$$h_3 = \tanh(0.5(0.2) + 0.8(0.6164) + 0.1) \approx 0.6000$$

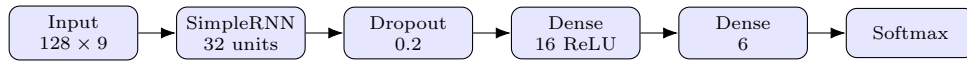
The values obtained manually were compared with the program output in the notebook, and the results were consistent:

$$h_1 \approx 0.3364, \quad h_2 \approx 0.6164, \quad h_3 \approx 0.6000$$

This demonstrates how the hidden state at each time step depends on both the current input and the hidden state from the previous time step.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

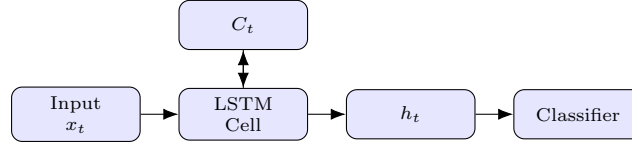
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference:

The cell state of LSTM works like a long-term memory channel where valuable information is passed from one time step to another. The role of the forget gate is to decide what previous information needs to be forgotten, whereas the input gate decides on the new information that is to be stored in the cell state. The output gate decides on what part of the stored information is made available as the hidden state.

Hence, the gates help control the flow of information through the cell state in such a way that valuable information is retained, whereas unnecessary information is eliminated.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

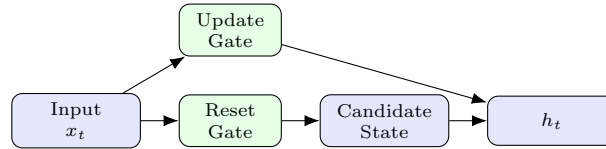
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

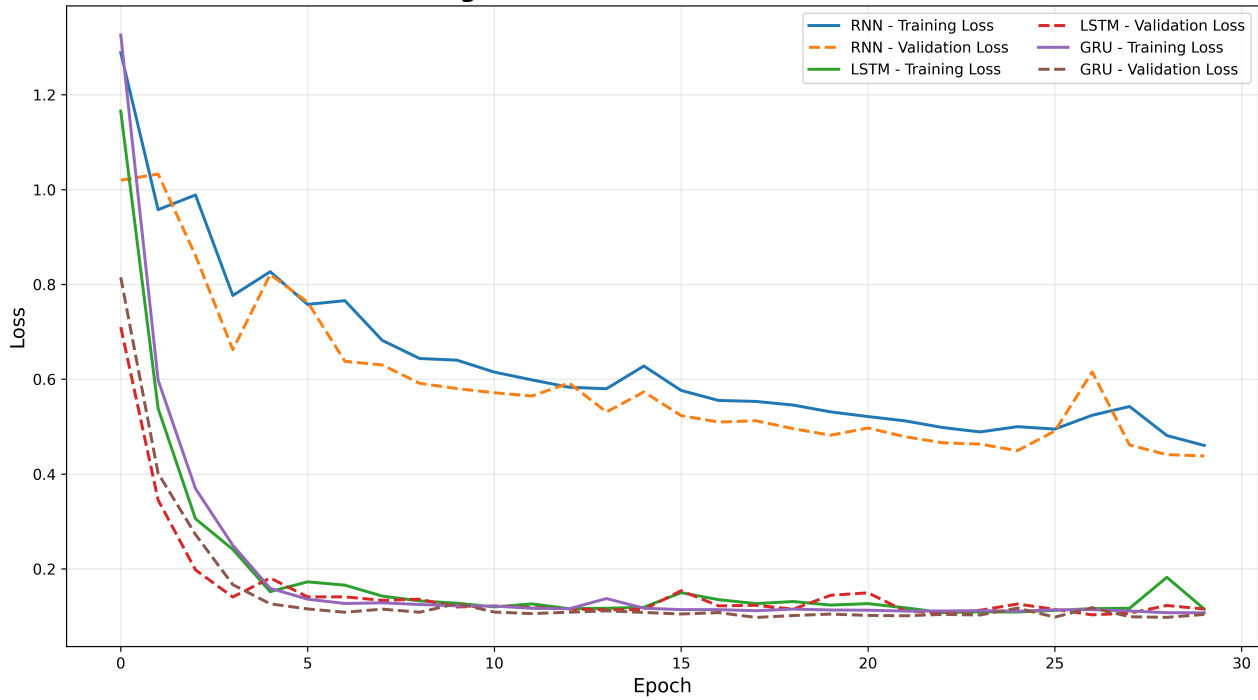
Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

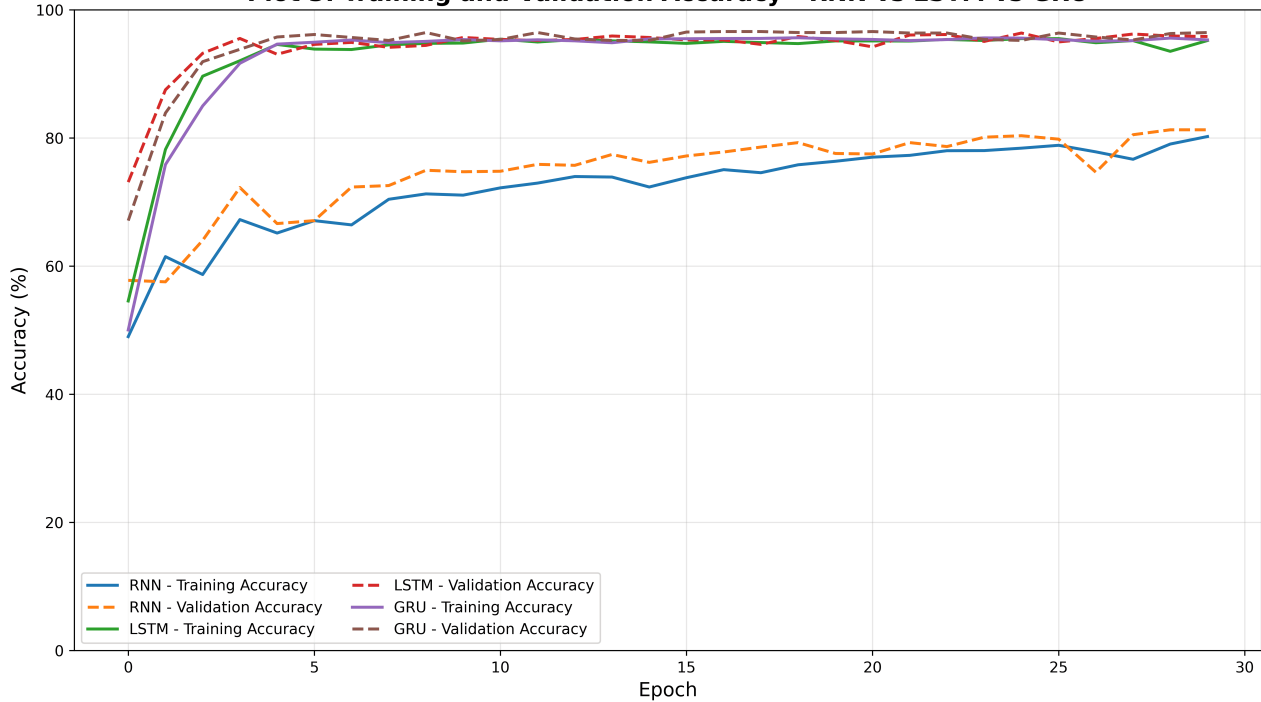
13. Required Training Plots

For each of the RNN, LSTM and GRU models, the training and validation performance was plotted over 30 epochs.

Plot 2: Training and Validation Loss - RNN vs LSTM vs GRU



Plot 3: Training and Validation Accuracy - RNN vs LSTM vs GRU



Inference for Plot 2 – Training and Validation Loss:

Loss in training and validation decreases as the models keep learning until they get stable, showing that they have converged. Difference between the curves denotes the ability of the model to generalize to the unseen validation data. A small and stable difference shows good generalization.

Inference for Plot 3 – Training and Validation Accuracy:

The accuracy on training and validation sets usually rises while the model learns and then stabilizes at some point, showing that it converged. When the accuracy on both training and validation sets is similar, the model learns generalizable patterns. Otherwise, there will be a generalization gap.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

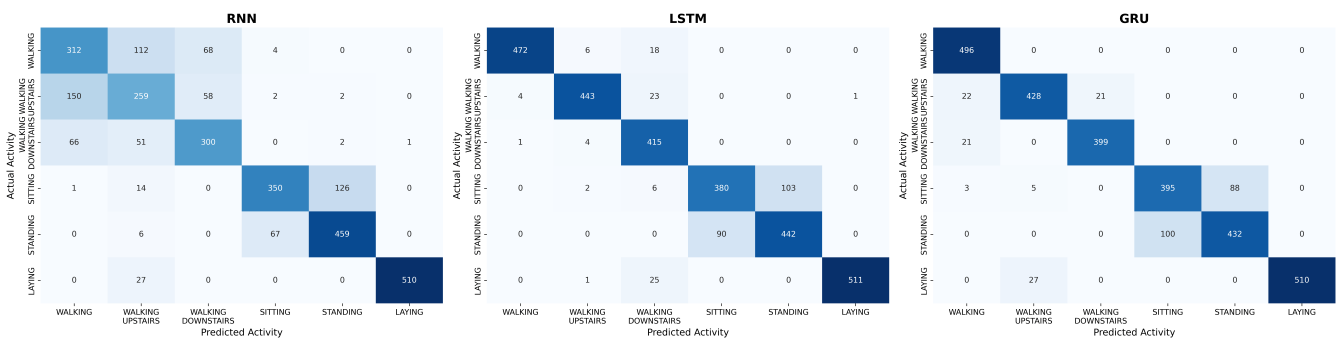
Metric	RNN	LSTM	GRU
Accuracy (%)	74.31	90.36	90.26
Macro Precision (%)	74.18	90.51	90.41
Macro Recall (%)	73.64	90.61	90.42
Macro F1 (%)	73.79	90.45	90.36
Parameters	1,974	6,006	4,758
Training Time (s)	68.70	77.67	75.22

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

The confusion matrices for the RNN, LSTM and GRU models are shown below.

Plot 4: Confusion Matrices - RNN vs LSTM vs GRU



Inference:

The confusion matrices are indicative of the activity-wise classification accuracy of the three recurrent models. The elements on the main diagonal indicate the correctly classified examples, whereas the elements off the main diagonal

denote misclassifications.

- **Highest recognition rate:** LAYING and WALKING activities are accurately detected by all models with significantly large diagonal elements.
- **Largest number of misclassifications:** The majority of errors belong to the SITTING and STANDING categories, because of their similar sensor data and higher difficulty in their differentiation.
- **Activities frequently confused with one another:** SITTING and STANDING form the most prominent confusion pair. The confusion is also present between walking activities WALKING, WALKING UPSTAIRS and WALKING DOWNSTAIRS.
- **Consistency across models:** The pattern of errors is the same for RNN, LSTM and GRU neural networks, but the amount of errors is smaller in case of LSTM and GRU, which corresponds to better test results of these two architectures.

Therefore, according to the confusion matrices, the recurrent models successfully classify the dynamic activities but struggle with classifying stationary activities with similar sensor data.

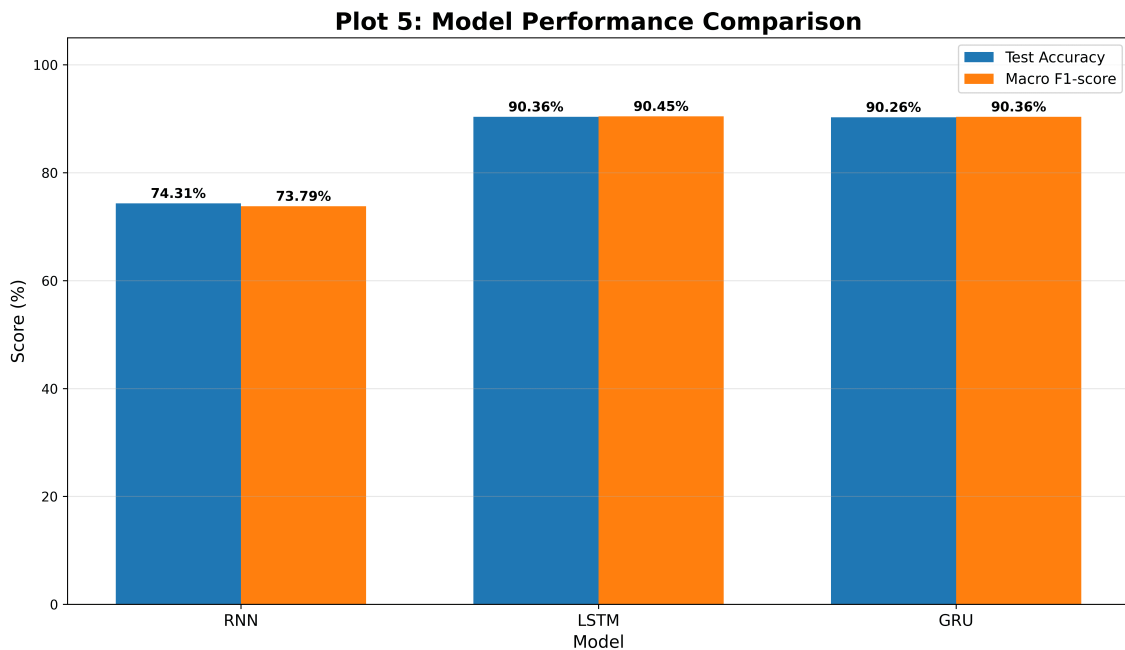
16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	68.70	77.67	75.22
Test F1-score (%)	73.79	90.45	90.36

Plot 5: Model Performance Comparison

The test Accuracy and Macro F1-score of the RNN, LSTM and GRU models are compared in the following plot.



Inference:

The plots illustrate the predictability of all three RNN models. The RNN model provides a 74.31% test accuracy and 73.79% Macro F1-score, whereas the LSTM model gives a 90.36% test accuracy and 90.45% Macro F1-score. The GRU gives a 90.26% test accuracy and 90.36% Macro F1-score.

The number of parameters for the LSTM is 6,006, and its training duration is 77.67 seconds, while for the GRU it is 4,758 and 75.22 seconds correspondingly. The smallest size of the model belongs to the RNN, which has 1,974 parameters and 68.70 seconds of training.

Therefore, the LSTM and GRU provide considerably better predictability than the RNN does, with the additional complexity and cost of training. The GRU has a similar level

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	73.26	81.57	85.58
64	73.57	88.36	89.30
128	73.79	90.45	90.36

Plot 6: Sequence Length vs. Test F1-score



Inference:

It is found that increasing the sequence length increases the Macro F1-score, especially for LSTM and GRU. While the improvement in the F1-score for RNN is marginal, from 73.26% when the length of the sequence is 32 to 73.79% for 128 time steps, the F1-scores for LSTM and GRU increase significantly to 90.45% and 90.36% respectively.

The reason for such an improvement is that increasing the length of the sequence allows more temporal information, which helps the LSTM and GRU to capture the patterns of activities.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```

Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
  
```

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

The pretrained MobileNetV2 model was used as a frozen CNN feature extractor. With `include_top=False` and global average pooling, the CNN produces a feature vector of dimension

$$D = 1280$$

for each video frame. Since 10 frames are sampled uniformly from each video, the recurrent network receives a sequence of 10 CNN feature vectors:

$$X \in \mathbb{R}^{10 \times 1280}$$

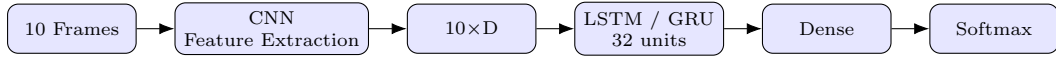
For the complete set of 60 videos used in the experiment, the input tensor supplied to the recurrent network is therefore:

$$X \in \mathbb{R}^{60 \times 10 \times 1280}$$

where 60 is the number of videos, 10 is the number of sampled frames per video, and 1280 is the MobileNetV2 feature dimension.

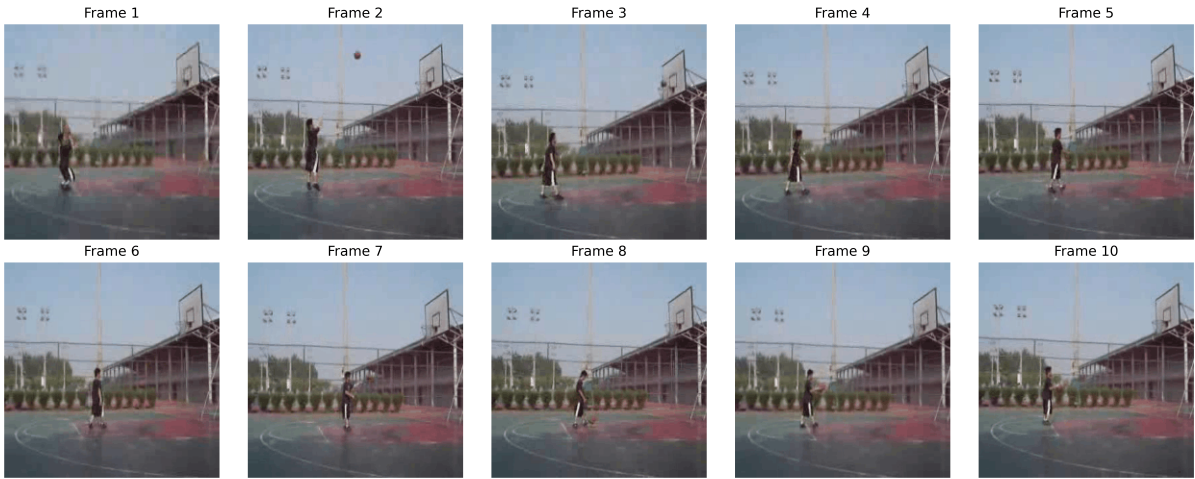
21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



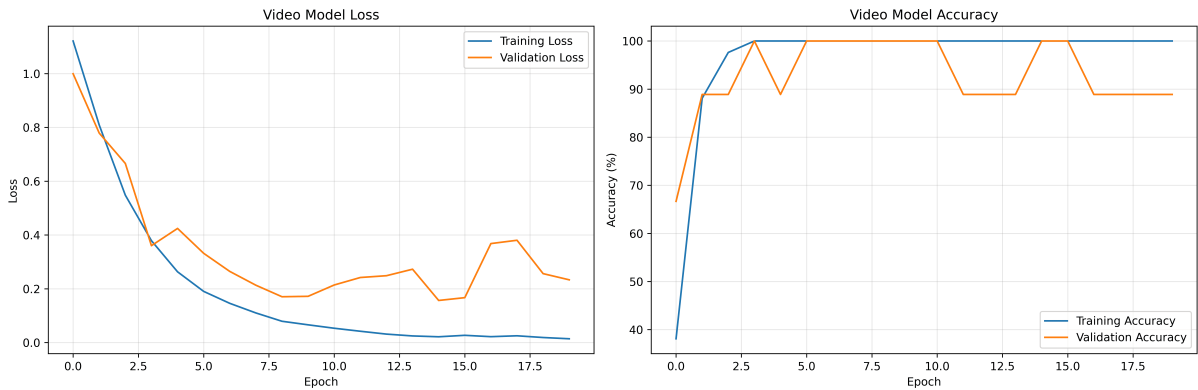
Plot 7: Video Sample Frames

A representative sequence of 10 uniformly sampled frames from a video is shown below.



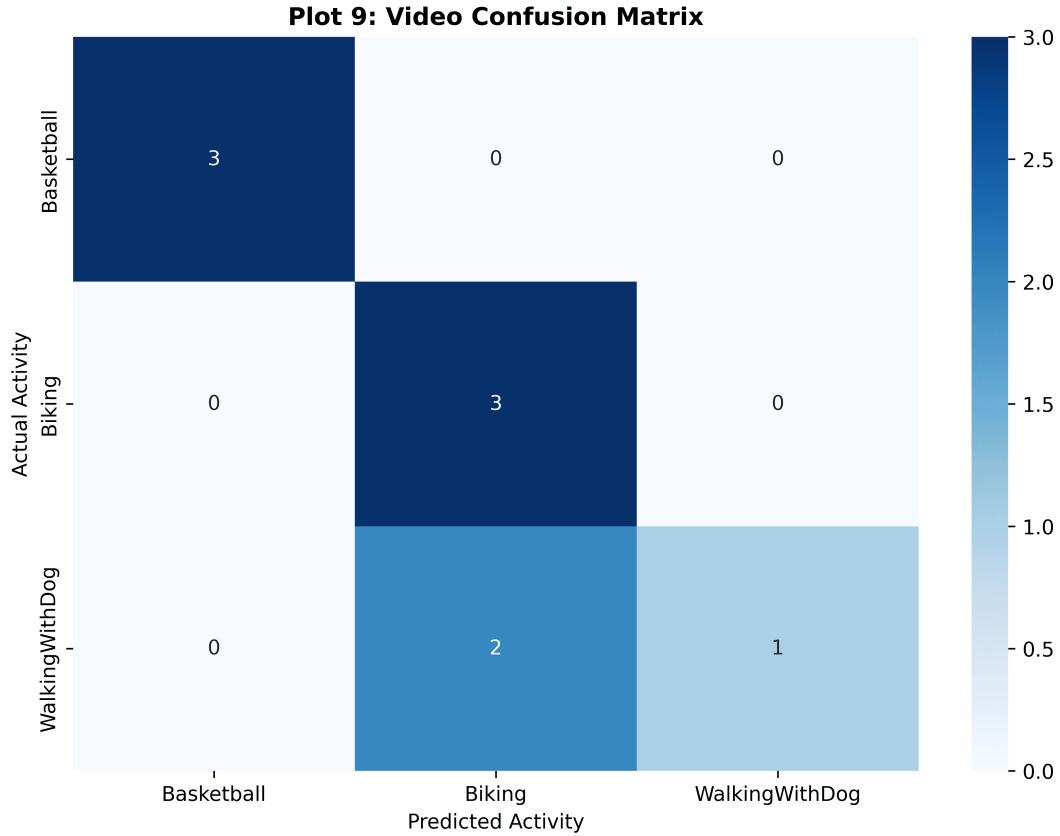
The CNN detects the spatial details in each frame including objects, shape, texture, and visual patterns. The detected details of each frame are arranged in a series form and fed to the LSTM. Temporal relations between subsequent frames are learned by the recurrent network that classifies the video activity based on those.

Plot 8: Video Training and Validation Curves



These plots provide the learning characteristics of the CNN-LSTM based video classification model. The learning plots show how the model learns from the CNN extracted features in training epochs. The gap between the training and validation shows the generalization gap that may indicate the presence of overfitting.

Plot 9: Video Confusion Matrix



From the confusion matrix, it can be seen that all the three Basketball and Biking videos have been classified correctly. However, in the case of WalkingWithDog, 1 video has been correctly classified, but 2 videos have been classified as Biking. Therefore, there has been confusion in the classification of WalkingWithDog and Biking.

The test accuracy and Macro F1 score obtained by the recurrent classifier was 77.78% and 75.00%, respectively. It can be inferred from the confusion that the temporal and visual features of WalkingWithDog were similar to that of Biking in some cases.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[8,4,7,3,9]	[9,3,7,4,8]
2	[10,5,10,4,1]	[1,4,10,5,10]
3	[10,3,6,8,9]	[9,8,6,3,10]
4	[6,1,3,7,8]	[8,7,3,1,6]
5	[9,4,6,2,4]	[4,2,6,4,9]

The predicted outputs exactly match the reversed input sequences for all five test examples. This shows that the encoder-decoder LSTM successfully learned the sequence reversal task. The model achieved a Token Accuracy of 100.00% and a Sequence Accuracy of 100.00%.

24. Sequence-to-Sequence Evaluation

The trained encoder-decoder LSTM was evaluated using token accuracy and sequence accuracy. The final training and validation losses were also recorded.

- **Token Accuracy:** 100.00%
- **Sequence Accuracy:** 100.00%
- **Final Training Loss:** 0.00094
- **Final Validation Loss:** 0.0011

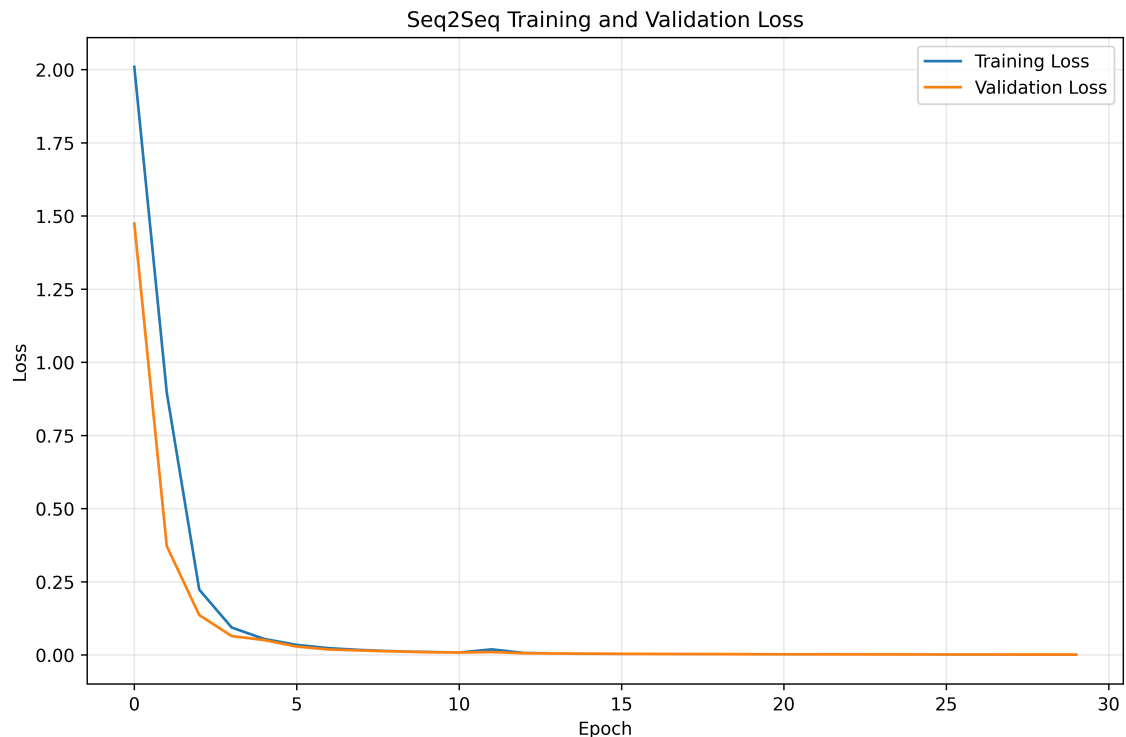
Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Plot: Seq2Seq Training and Validation Loss



Inference:

Token accuracy is concerned with the accuracy of individual components, while sequence accuracy needs all the tokens in a sequence to be accurate. This implies that any mistake in the token level will cause a mistake in the sequence level, and sequence accuracy will be very low compared to token accuracy. In the current experiment, token and sequence accuracy was 100.00%.

25. Consolidated Results

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	74.31	74.18	73.64	73.79	1,974
LSTM	90.36	90.51	90.61	90.45	6,006
GRU	90.26	90.41	90.42	90.36	4,758
CNN-LSTM/GRU	77.78	86.67	77.78	75.00	169,219

26. Required Inferences

1. Temporal Pattern in Sensor Signals

From the temporal sensor plots, we can see that there are repeated fluctuations in WALKING and WALKING UPSTAIRS due to constant body movement. On the other hand, SITTING depicts relatively stable signals. Ordering of the 128 readings is crucial as it captures the temporal motion pattern needed for sequence classification.

2. Convergence Behavior of RNN

As seen from the graph depicting training and validation of RNN, it can be said that the learning takes place during the early epochs and converges to a stable area. The test accuracy and Macro F1-Score obtained from the model is 74.31% and 73.79%, respectively.

3. Convergence Behavior of LSTM

The training and validation plots for LSTM reveal the process of learning followed by convergence. LSTM model achieved an accuracy of 90.36% on testing data and 90.45% Macro F1-score. Cell states and gates of LSTM enable it to retain information along the sequence.

4. Convergence Behavior of GRU

As seen from the plots below, GRU shows that the model is learning and gradually converging throughout training. The GRU gave us 90.26% accuracy and 90.36% Macro F1-Score. With fewer parameters than the LSTM, GRU has update and reset gates that enable it to control the information flow.

5. Overfitting and Underfitting

Training and validation curves were employed in analyzing the generalization gap. A significant increase in the gap between training and validation results would be an indication of overfitting, while low performance without much improvement would be an indication of underfitting. The two curves were thus analyzed for the generalization behavior of the models during training.

6. Activity-wise Errors from the Confusion Matrix

It can be seen from the confusion matrices that the models correctly classify the activities on the diagonal, but errors appear where the activity sensor signals are similar. There is confusion between the SITTING and STANDING, and there is also confusion between the walking activities. The confusion matrices of the LSTM and GRU have less errors compared to the RNN matrix.

7. Effect of Sequence Length

Increasing the sequence length can enhance the Macro F1-score, particularly when using LSTM and GRU models. For example, an RNN F1-score varies from 73.26% at 32 time steps to 73.79% at 128 time steps. However, an improvement from 81.57% to 90.45% is observed for LSTM, and an increase from 85.58% to 90.36%.

8. Parameter and Computational Differences

The RNN model has 1,974 parameters with a total time taken to train of 68.70 seconds. LSTM has 6,006 parameters that take 77.67 seconds to train, whereas the GRU takes 4,758 parameters to train which take 75.22 seconds. This is because the extra gates in LSTM and GRU increase the complexity and cost of training.

9. Role of CNN in Video Understanding

The pretrained CNN architecture called MobileNetV2 performs the task of extracting spatial features from individual

video frames. The individual video frames are encoded into 1,280-dimensional vectors, resulting in a matrix of size 10×1280 .

10. Role of the Recurrent Model in Temporal Modeling

The recurrent neural network takes in the input sequence of CNN feature vectors and learns how the visual features evolve through the selected frames. Therefore, the CNN model extracts spatial information from the individual frames, whereas the LSTM model captures the temporal relations between the frames. The video classification model yielded 77.78% accuracy and 75.00% Macro F1-score.

11. Token Accuracy vs. Sequence Accuracy

Accuracy of token prediction is dependent on the correctness of individual output tokens, while accuracy of sequence prediction depends on the correctness of the whole sequence. Hence, an individual sequence may have most of its output tokens correct, yet still be considered wrong if at least one token was not predicted correctly. In the present experiment, the accuracy of both types of predictions was 100.00%.

27. Discussion Questions

1. What is a recurrent neural network?

A Recurrent Neural Network (RNN) is a neural network designed for sequential data. Unlike a feed-forward network, it maintains a hidden state that carries information from previous time steps. At time step t , the hidden state is updated using the current input and the previous hidden state:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

This allows the network to use both the current observation and information from earlier observations when making predictions.

2. What information is represented by the hidden state?

The hidden state is a learned representation of the sequence data that has been observed till the current time step. The hidden state can be thought of as a short-term memory, which stores information from the past inputs relevant to the current input.

3. Why is sequence order important?

Values can be organized in various sequences that could indicate totally new temporal sequences. For instance, in activity recognition, the ordering of sensors can reflect the dynamics of body motion with time. Hence, preservation of temporal order enables the model to learn about motion dynamics as opposed to independent observations.

4. Explain the difference between a feed-forward network and an RNN.

The feed-forward network uses an input signal without retaining any information about previous inputs. The RNN retains its hidden state and transmits information across time steps. Thus, feed-forward networks are best suited for independent inputs, while RNNs are specifically designed for modeling dependencies in sequential inputs.

5. Explain Backpropagation Through Time.

BPTT (Back Propagation Through Time) is the algorithm employed for training in an RNN. The recurrent neural network is effectively unrolled over the time steps, and the error is backpropagated over the time steps. Due to the fact that the same recurrent weights are employed over all time steps, the gradients from all time steps help update the shared weights.

6. Why can gradients vanish during BPTT?

In BPTT, gradients will multiply across the recurrent connections and derivative of activations over and over again. In cases where such numbers are always less than one, the gradients may become infinitesimally small when trying to move backward to earlier time-steps. Hence, learning of long-term dependencies becomes very hard.

7. Why can gradients explode during BPTT?

Conversely, the opposite problem happens when multiplying the gradients repeatedly causes the values to be greater than one. When this happens, the gradients increase very quickly as they propagate over a long sequence of time steps. This problem is known as the exploding gradient problem.

8. What are long-term dependencies?

Long term dependencies arise where the information necessary for making a prediction at the present time step relies on data from long ago in the sequence. It is challenging for the simple RNN to learn this dependency since information and gradients can get lost after many recursive steps.

9. What is the role of the LSTM cell state?

The memory cell state of an LSTM is a separate channel for the conveyance of information over time. The gates of an LSTM determine whether the information should be discarded, whether any new information should be added to the cell state, and whether any information should be passed to the hidden state.

10. **What are the forget, input and output gates in LSTM?**

The **forget gate** decides the amount of information to forget from the old memory state. The **input gate** decides what new information should be added to the memory state. The **output gate** decides the amount of updated memory state to expose as the hidden state.

Hence, these three gates control the flow of information in the LSTM memory.

11. **What are the reset and update gates in GRU?**

The **reset gate** is responsible for deciding how much of the previous hidden state should be utilized to create the new hidden state. The **update gate** is responsible for deciding how much of the previous hidden state to keep and how much new information to use.

12. **Compare LSTM and GRU.**

Both LSTM and GRU are gated RNNs developed to address the problem of handling long-term dependencies better than a traditional RNN can. While the LSTM architecture maintains the hidden and cell state along with forget, input, and output gates, GRU utilizes the hidden state alone with reset and update gates. Hence, GRU is structurally simpler than the LSTM.

13. **Why does GRU generally use a simpler gating mechanism than LSTM?**

The GRU has a few of the responsibilities of the LSTM gates combined within itself but does not have an independent cell state. The update and reset gates regulate the flow of information with a less complicated design. This makes it easier for the neural network to perform its tasks.

14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**

For a meaningful comparison between these models, the only difference between them should be in the recurrent architecture itself. With the use of the same dataset, preprocessing, input representation, output layer, optimizer, batch size, number of epochs, and evaluation method, the performance difference will be attributed more effectively to the recurrent architecture.

15. **What does the confusion matrix reveal that accuracy alone does not?**

However, accuracy is used to represent the general ratio of correctly classified instances, but accuracy fails to tell us which categories are being confused. In a confusion matrix, correct classification appears on the diagonals while incorrect classification appears on the non-diagonal blocks of the matrix.

16. **Why should macro F1-score be reported for multi-class classification?**

Macro F1-Score first computes the F1-Score for every individual class and then averages it. This is because the macro F1-score gives equal weight to each class. Hence, it is better when there is imbalance between the number of samples or performance across classes.

17. **What is the significance of sequence length?**

The temporal context depends on the length of the sequence. Shorter sequences may contain missing details about the activity while long sequences will have a better representation of the temporal patterns. Long sequences however entail more computational effort and may contain irrelevant details.

18. **Why can a CNN be used as a feature extractor for video?**

The CNN model is able to learn from the spatial features in images. In the video, the spatial feature includes the shape, object, texture and visual structure in the frames. The pretrained CNN model can thus transform each frame into a feature vector without training from scratch for the small video dataset.

19. **What type of information is extracted by the CNN?**

The CNN captures spatial and visual information in the individual images. These patterns include those pertaining to objects, shape, texture, edges, and many more visual features. In this case, MobileNetV2 converts each image to a feature vector with 1280 dimensions.

20. **What type of information is learned by the recurrent network?**

The recurring network recognizes the relationships between the features extracted by the CNN through time. This is done by observing the changes in the visual representation from one frame to the next.

21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**

A video is made up of several frames that follow a certain order in time. The CNN converts each frame into a feature vector, which is then ordered in sequence to maintain the temporal order so that the LSTM or GRU can learn about the change in spatial data in the video.

22. **Explain the encoder and decoder in sequence-to-sequence learning.**

Encoder: Input sequence is read by the encoder, which encodes the sequence and stores the representation in such a way that it carries information about the sequence. **Decoder:** Decoder uses this representation to produce the output sequence. Hence, encoder learns the input sequence, whereas decoder generates the output sequence.

23. **What is teacher forcing?**

Teacher forcing is a method of training where the decoder gets the correct previous output as its input in the next time step instead of taking its own prediction from the previous time step. This helps in providing a better learning signal during training.

24. **Differentiate token accuracy and sequence accuracy.**

Token accuracy measures the percentage of individual tokens predicted correctly:

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy measures the percentage of complete sequences for which every token is correct:

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Therefore, one incorrect token can make an entire sequence incorrect even when the remaining tokens are correct.

25. Why must the test set remain untouched during model selection?

The test set is designed in order to give an impartial estimate of the performance of the final model. The use of the same test set to pick models, hyperparameters, and any pre-processing is indirect usage of the information contained within the test set, which leads to the phenomenon of data leakage.

28. Additional Exercises

1. Effect of Number of Recurrent Units

The number of recurrent units was varied between 16 and 64 for the RNN, LSTM and GRU models. The measured test performance did not show a consistent improvement when the number of units was increased.

Model	Units	Accuracy (%)	Macro F1 (%)
RNN	16	35.87	43.30
LSTM	16	46.93	43.64
GRU	16	50.83	29.33
RNN	64	41.17	36.33
LSTM	64	48.38	31.89
GRU	64	51.82	43.84

These results show that increasing the number of recurrent units did not automatically improve the measured official-test performance in these additional runs.

2. LSTM vs GRU with 32 Units

A controlled comparison was performed using 32 recurrent units for both LSTM and GRU.

Metric	LSTM	GRU	Difference
Accuracy	87.00%	89.01%	2.01 pp
Macro F1	86.78%	88.96%	2.18 pp
Parameters	6,006	4,758	1,248 fewer
Training Time	20.70 s	19.89 s	0.81 s

In this experiment, the GRU used fewer parameters and obtained higher measured test accuracy and Macro F1-score than the LSTM.

3. Two Recurrent Layers

Two recurrent layers with 32 units each were tested to investigate the effect of increasing the depth of the recurrent network.

Model	Accuracy (%)	Macro F1 (%)	Parameters
2-layer RNN	33.05	25.89	4,054
2-layer LSTM	46.18	35.84	14,326
2-layer GRU	55.62	46.19	11,094

The deeper recurrent models did not improve the official-test performance relative to the corresponding one-layer baseline in this experiment.

4. Bidirectional LSTM

A bidirectional LSTM with 32 units was compared with the unidirectional LSTM.

Model	Accuracy (%)	Macro F1 (%)	Parameters
LSTM	87.00	86.78	6,006
BiLSTM	42.25	31.28	11,894

The BiLSTM had more parameters but showed substantially lower official-test performance in this particular run. Therefore, this result is treated as an observation from the given experimental setup rather than a general conclusion about bidirectional LSTMs.

5. Sequence Length and Computational Cost

The measured training times for different sequence lengths were:

Sequence Length	RNN	LSTM	GRU
32	23.46 s	11.33 s	15.35 s
64	17.58 s	16.39 s	16.62 s
128	23.74 s	19.46 s	18.06 s

The recorded training times were affected by execution and batching overhead, so they were not perfectly monotonic. However, longer sequences still required the recurrent models to process more time steps.

6. CNN-LSTM vs CNN-GRU

The two video recurrent models were compared using the same CNN feature extractor.

Metric	CNN-LSTM	CNN-GRU
Accuracy	96.43%	89.29%
Macro F1	96.19%	88.85%
Parameters	168,643	126,723
Training Time	10.30 s	10.30 s

CNN-LSTM obtained higher measured test performance in this three-class experiment, while CNN-GRU used fewer parameters.

7. Different Input and Output Lengths

The sequence-to-sequence task was modified so that the input sequence contained four tokens while the output sequence contained five tokens.

The model achieved:

Metric	Result
Token Accuracy	85.52%
Sequence Accuracy	30.53%
Completely Correct Test Sequences	229 / 750

The difference between token accuracy and sequence accuracy demonstrates the stricter nature of complete-sequence evaluation. A sequence is counted as correct only when all its output tokens are predicted correctly.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder-decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," *EMNLP*, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," *ESANN*, 2013.

5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>